

Exception Handling

COMP1039 Programming Paradigms

Yuan Cheng

Spring 2026

School of Computer Science
The University of Nottingham Ningbo China

Exception Handling

Quick poll:

- 1 = compile-time error
- 2 = runtime exception
- 3 = no error

```
1  int n = "Integer";  
2  
3  int[] xs = new int[3];  
4  xs[7] = 7;
```

Compile-time vs Runtime

Compile-time error

- Detected by the Java compiler.
- Program cannot run until fixed.
- Example: `int n = "Integer";`

Runtime exception

- Detected by JVM during execution.
- Program can compile, but may crash while running.
- Examples: array index out of bounds, divide by zero.

Exception Hierarchy

- Root class: `java.lang.Throwable`
- Two direct subclasses:
 - `Error`: serious JVM/system-level issues (normally not handled in app code)
 - `Exception`: issues from program logic or external conditions
- We focus on `RuntimeException`, a key subclass under `Exception`.

Checked vs Unchecked Exceptions

Unchecked

- `RuntimeException`, `Error`, and their subclasses.
- Usually programming logic bugs that are not recoverable.
- Can occur anywhere in the program.
- Compiler does not force you to catch.

Checked

- All other exceptions (e.g., `IOException`).
- Compiler forces you to catch or declare with `throws`.

Common Unchecked Exceptions

Exception	Typical cause
<code>ArithmeticException</code>	division by zero
<code>NullPointerException</code>	dereferencing null
<code>NumberFormatException</code>	invalid numeric parsing
<code>ClassCastException</code>	invalid cast
<code>ArrayIndexOutOfBoundsException</code>	invalid array index
<code>StringIndexOutOfBoundsException</code>	invalid string index

Five Exception Handling Keywords

- `try`: monitor code that may throw exceptions.
- `catch`: handle a specific exception type.
- `throw`: manually throw an exception object.
- `throws`: declare exceptions a method may pass to caller.
- `finally`: always-run cleanup block when leaving a try/catch block.

try and catch

```
1 try {  
2     // code that may throw exception  
3 } catch (ExceptionType1 e) {  
4     // handle exception type 1  
5 } catch (ExceptionType2 e) {  
6     // handle exception type 2  
7 }
```

Rule: `catch` cannot exist without `try`.

Example 1: Catching Array Index Error

```
1  class ExcDemo1 {
2      public static void main(String[] args) {
3          int[] nums = new int[4];
4          try {
5              System.out.println("Before");
6              nums[7] = 10;
7              System.out.println("After");
8          } catch (ArrayIndexOutOfBoundsException e) {
9              System.out.println("Index out of bounds!");
10         }
11         System.out.println("Catch finishes");
12     }
13 }
```

Example 2: Exception in Method Call

```
1  class ExcDemo2 {
2      static void genException() {
3          int[] nums = new int[4];
4          nums[7] = 0;
5      }
6      public static void main(String[] args) {
7          try {
8              genException();
9          } catch (ArrayIndexOutOfBoundsException e) {
10              System.out.println("Index out of bounds!");
11          }
12          System.out.println("Catch finishes");
13      }
14 }
```

Example 3: Continue After Exception

```
1  class ExcDemo3 {
2      public static void main(String[] args) {
3          int[] nums = {4, 0, 8, 16};
4          int total = 0;
5          for (int n : nums) {
6              try {
7                  System.out.println(80 / n);
8                  total += 80 / n;
9              } catch (ArithmeticException e) {
10                 System.out.println("Cannot divide by zero");
11             }
12         }
13         System.out.println("Total = " + total);
14     }
15 }
```

Multiple catch Blocks

- It is possible to catch multiple exceptions by using more than one `catch` clauses.
- But each `catch` must catch a different type of exception.

```
1  int[] nums1 = {4, 2, 8, 16};
2  int[] nums2 = {0, 3};
3
4  for (int i = 0; i < nums1.length; i++) {
5      try {
6          System.out.println(nums1[i] / nums2[i]);
7      } catch (ArithmeticException e) {
8          System.out.println("Cannot divide by zero");
9      } catch (ArrayIndexOutOfBoundsException e) {
10         System.out.println("Index out of bounds");
11     }
12 }
```

Nested Calls and Propagation

```
1 static void method3() { int x = 4 / 0; }
2 static void method2() {
3     method3();
4     System.out.println("After m3");
5 }
6 static void method1() {
7     method2();
8     System.out.println("After m2");
9 }
10 public static void main(String[] args){
11     try {
12         method1();
13         System.out.println("After m1");
14     } catch(ArithmeticException e){
15         System.out.println("Exception in main");
16     }
17 }
```

Question:

If only main has try-catch, where is the exception handled?

throw: Manually Throwing Exception

```
1 try {  
2     System.out.println("Before");  
3     throw new ArithmeticException();  
4 } catch (ArithmeticException e) {  
5     System.out.println("Exception caught");  
6 }
```

Rethrowing to Outer Block

```
1  try {
2      try {
3          throw new ArithmeticException();
4      } catch (ArithmeticException e) {
5          System.out.println("Inner catch");
6          throw e; // rethrow
7      }
8  } catch (ArithmeticException e) {
9      System.out.println("Outer catch");
10 }
```

finally: Always-run Block

```
1 try {
2     System.out.println("Before");
3     throw new ArithmeticException();
4 } catch (ArithmeticException e) {
5     System.out.println("Exception caught");
6     return;
7 } finally {
8     System.out.println("Cleanup in finally");
9 }
```

Key idea: `finally` runs even when `return` appears in `catch`.

Trace a Program I: No Exception

```
1 try {  
2     statements;  
3 } catch (ExceptionType ex) {  
4     handle ex;  
5 } finally {  
6     finalStatement;  
7 }  
8 nextStatement;
```

- Execute `try` block.
- Skip `catch`.
- Execute `finally`.
- Continue to `nextStatement`.

Trace a Program II: Exception Handled

```
1 try {
2     statement1;
3     statement2; // throws ExceptionType
4 } catch (ExceptionType ex) {
5     handle ex;
6 } finally {
7     finalStatement;
8 }
9 nextStatement;
```

- Jump from failing statement to matching `catch`.
- Execute `finally`.
- Continue after the block.

Trace a Program III: Exception Rethrown

```
1  try {  
2      statement1;  
3      statement2; // throws ExceptionType  
4  } catch (ExceptionType ex) {  
5      handle ex;  
6      throw ex;  
7  } finally {  
8      finalStatement;  
9  }  
10 nextStatement; // skipped in this method
```

- `catch` runs, then rethrows.
- `finally` still runs.
- Control transfers to caller's handler.

throws for Checked Exceptions

```
1 type methodName(params) throws ExceptionType {  
2     // method body  
3 }
```

- If a checked exception is not handled inside a method, declare it.
- Unchecked exceptions do not require declaration.

Checked Exception Example: IOException

```
1 public static char prompt(String str) throws java.io.IOException
   {
2     System.out.println(str + ": ");
3     return (char) System.in.read();
4 }
```

- `System.in.read()` may throw `IOException`.
- Compiler requires handling or declaration.

Caller Handles It

```
1 public static void main(String[] args) {
2     char ch = 'X';
3     try {
4         ch = prompt("Enter a letter");
5     } catch (java.io.IOException e) {
6         System.out.println("I/O exception occurred.");
7     }
8     System.out.println("You pressed " + ch);
9 }
```

Use `throws` when:

- handling is better done by the caller.
- method cannot recover meaningfully at this level.

Use `try` and `catch` when:

- you can recover locally (retry, default value, fallback).
- you want to prevent abrupt termination and continue.

Multi-catch (JDK 7+)

```
1 try {  
2     // risky code  
3 } catch (ArithmeticException | ArrayIndexOutOfBoundsException e)  
4 {  
5     System.out.println("Math or index issue");  
6 }
```

- Reduces duplication when handling logic is identical.
- Exception variable in multi-catch is implicitly final.

Live Trace Activity: Predict the Output

```
1 public class TraceDemo {  
2     public static void main(String[] args) {  
3         try {  
4             System.out.println("A");  
5             int x = 10 / 0;  
6             System.out.println("B");  
7         } catch (ArithmeticException e) {  
8             System.out.println("C");  
9         } finally {  
10            System.out.println("D");  
11        }  
12        System.out.println("E");  
13    }  
14 }
```

-
1. A B C D E
 2. A C D E
 3. A C E
 4. A D E

Live Trace Activity: Variant

Now consider this slightly modified version.

```
1 public class TraceDemo2 {  
2     public static void main(String[] args) {  
3         try {  
4             System.out.println("A");  
5             System.out.println("B");  
6         } catch (ArithmeticException e) {  
7             System.out.println("C");  
8         } finally {  
9             System.out.println("D");  
10        }  
11        System.out.println("E");  
12    }  
13 }
```

Question: What changed compared with the previous example?

- Does the `catch` block run?
- Does the `finally` block still run?
- What is the new output order?

Exit Slips

- Take **1–2 minutes** to reflect on this lecture.
- On a sheet of paper, write:
 - **One thing you learned** in this lecture
 - **One thing you didn't understand**